1CSmallControl — аудит безопасности, качества кода и производительности

Документ-рекомендация и готовый промт для ИИ-агента. Дата: 8 ноября 2026 г. Репозиторий: `E:\_Project\1CSmallControl`, ветка `main`, коммит `fbbc774` (версия 2.0.0, 10 локальных коммитов не отправлены). Сервер развёртывания: `192.168.0.5` (`1c.dgek.ru`).

> **Режим аудита.** На сервере выполнялись только команды чтения от непривилегированного пользователя `dgek` (ключ `id_ed25519_dgek1c` не авторизован для `root`; `sudo` требует пароль). Ничего не изменялось: ни код, ни конфигурация, ни версия, ни git. Разделы, требующие `root` (точное содержимое `sudoers.d`, правила `ufw`, `postgresql.conf`, `pg_hba.conf`, права на `/opt/1c-portal` и `/etc/1c-portal`), помечены как «проверить с root».

Содержание

1. Итог: главные находки
2. Промт для ИИ-агента (готов к использованию)
3. Инвентаризация: код и сервер
4. Безопасность (S-01…S-20)
5. Качество кода и оптимизация (C-01…C-12)
6. Производительность: портал и сервер (P-01…P-11)
7. План внедрения по этапам
8. Проверки и приёмка
9. Что проверить с root
10. Приложение: измерения и команды

1. Итог: главные находки

Портал в целом спроектирован аккуратно: пароли PBKDF2, CSRF-токены во всех формах, параметризованный SQL, `subprocess` без `shell=True`, cookie `Secure/HttpOnly/SameSite`, защита от подбора, 2FA, шлюз публикаций 1С с проверкой прав. Тем не менее найдены ошибки авторизации и границ привилегий, которые позволяют авторизованному *клиенту* выйти за пределы своей базы, а также архитектурные решения, снижающие производительность и надёжность на слабом CPU (Xeon L5520, 2,27 ГГц).

ID	Серьёзность	Находка	Суть исправления
S-08	Критично	Обход прав доступа к базам через шлюз `/WEB/<slug>/<rest>`: сегменты `..` не нормализуются, Apache 8081 схлопывает их. Подтверждено тестами на сервере (Werkzeug передаёт `rest='../test/'`, Apache отдаёт чужую публикацию 200).	Отклонять `.` / `..` / `//` / `\` в `rest`, формировать путь через `posixpath.normpath` и сверять префикс `/WEB/<slug>/`.
S-04	Высокий	Песочница: клиент может восстановить дамп *чужой* базы — проверяется только префикс каталога `/mnt/1cbackup/`.	Принимать только файлы из списка копий своей базы (по basename, regex, `realpath`).
S-03	Высокий	Клиент может включить «режим обновления» PostgreSQL: `fsync=off`, `full_page_writes=off`, `autovacuum=off` для *всего* кластера, без авто-возврата.	Только администратор; таймер авто-возврата; не отключать `fsync` / `full_page_writes`.

S-01	Высокий	Защита от подбора пароля обходится подделкой `X-Forwarded-For` (берётся первый элемент заголовка).	`ProxyFix(x_for=2)` и `request.remote_addr` везде.
S-02	Высокий	Веб-пользователь `www-data` (общий для Apache, модуля 1cws и портала) имеет sudo на `psql` от `postgres` → произвольный SQL суперпользователя.	Обёртка с фиксированным запросом, отдельный пользователь для gunicorn, sudo только ему.
S-05	Высокий	Песочница создаётся без `--scheduled-jobs-deny=on` (копия выполняет регламентные обмены/рассылки как боевая база); роли PostgreSQL `SUPERUSER`, md5.	Запрет регламентных заданий в копии; минимум привилегий роли; scram-sha-256.
C-02	Средний	Фоновый прогрев кэша сломан: `warm_counts()/warm_sizes()` вызываются без контекста приложения и молча падают → кэш размеров/состояний не прогревается.	`with app.app_context()`, убрать зависимость кэша от `request`, тест.
C-06	Средний	SQLite без WAL/ `busy_timeout` ; 7 событий «database is locked» за неделю; p90 кабинета клиента 6,1 с, p99 15 с (кратно 5-секундному ожиданию блокировки).	WAL, `busy_timeout`, индексы, ретенция журнала.
P-01	Средний	72 % всех HTTP-запросов (весь веб-клиент 1С) проходят через Python/gunicorn под GIL с таймаутом 300 с на поток.	nginx `auth_request` + прямой proxy к Apache, портал обслуживает только свои страницы.
S-10	Средний	Секреты в открытом виде: хук Mattermost в мир-читаемом unit-файле (прочитан без root), пароли SMB/автологина 1С/ приватные ключи WireGuard в SQLite и `settings.json`.	`EnvironmentFile` 0640, шифрование полей, ротация хука.

Полный список: 20 находок по безопасности, 12 по коду, 11 по производительности. Все ссылки на строки относятся к коммиту `fbbc774`.

2. Промт для ИИ-агента

Скопируйте блок целиком в чат агента (Copilot/Claude/GPT) вместе с этим документом. Промт описывает роль, ограничения, порядок работ и критерии приёмки.

```
РОЛЬ
Ты — старший инженер по безопасности и производительности Python/Flask + Linux (Ubuntu
24.04, Apache 2.4,
PostgreSQL Pro 1C 16, сервер 1С:Предприятие 8.3.27). Работаешь в репозитории
E:\_Project\1CSmallControl
```

```
(Flask-портал доступа к базам 1С: app.py ~3140 строк, maintenance.py, metrics.py, bash-
скрипты 1c-*.sh,
sudoers, deploy.sh, templates/, static/). Продакшен: 192.168.0.5 (1c.dgek.ru), HTTPS
терминирует nginx
на 192.168.0.15 → Apache :80 на .5 → gunicorn 127.0.0.1:8080 → Apache 127.0.0.1:8081
(публикации 1С, модуль wsap24).

ЖЁСТКИЕ ОГРАНИЧЕНИЯ
1. Ничего не применять на сервере 192.168.0.5 без отдельной команды пользователя. Разрешены
только
   read-only проверки. Любой деплой — только после фразы пользователя «применяй на сервер».
2. Не менять номер версии (PORTAL_VERSION, README, CHANGELOG «[Не выпущено]» можно
дополнять).
3. Не выполнять git push; коммиты делать локально, небольшими, с префиксами
fix(security)/perf/refactor.
4. Не печатать и не коммитить секреты (хуки Mattermost, пароли, ключи). При обнаружении —
маскировать.
5. Каждое исправление сопровождать тестом (pytest + app.test_client()) или воспроизводимой
проверкой.
6. Сохранять поведение UI; изменения в шаблонах — минимальные.

ПОРЯДОК РАБОТ (по убыванию приоритета; ID — из документа аудита docs/audit-recommendations-
2026-11-08.html)
Этап A — закрыть обход прав и границы привилегий:
  S-08 Шлюз /WEB/<slug>/<path:rest> (app.py:466-481): отклонять rest, содержащий сегменты
'.', '..',
       пустые сегменты ('//'), обратный слэш, управляющие символы; строить путь как
       posixpath.normpath('/WEB/'+slug+'/'+rest) и проверять startswith('/WEB/'+slug+'/');
при нарушении — 404
       через _web_denied(slug,'not-found'). Тесты: '/WEB/demo/../test/',
'/WEB/demo/..%2Ftest/',
       '/WEB/demo/%2e%2e/test/', '/WEB/demo/./ru_RU/' (последний допустим только после
нормализации в '/WEB/demo/ru_RU/').
  S-04 base_sandbox_create (app.py:2663-2735): принимать dump только если
os.path.basename(dump) входит в
       _sandbox_copies(row['db_name']) и realpath лежит в SANDBOX_DIR; regex r'^[a-z0-9_]+-
\d{8}-\d{4}\.dump$'.
  S-03 /portal/bases/<bid>/update-mode (app.py:2277-2298): убрать клиентский маршрут или
оставить только admin;
       в 1c-pg-update-mode.sh не трогать fsync/full_page_writes (оставить
synchronous_commit=off,
       maintenance_work_mem, max_wal_size, checkpoint_timeout); добавить systemd-timer
авто-возврата через N часов
       и уведомление в Mattermost при включении/возврате; в UI — предупреждение.
  S-01 _client_ip (app.py:293-298): удалить ручной разбор X-Forwarded-For;
ProxyFix(x_for=2, x_proto=1, x_host=1)
       (две доверенные прокси: nginx .15 и Apache на .5); везде request.remote_addr. Тест:
запрос с поддельным XFF
       не меняет IP в audit/login_fails.
  S-06 /jobs/<jid>/status и /jobs/active (app.py:3052-3080): клиент видит только задания с
user_id = свой uid
       или ref своих песочниц; admin — все.
  S-07 open redirect: безопасная проверка next в login (754-755) и login_2fa (2451):
```

```
urlsplit(next) без scheme/netloc,
       не начинается с '//' и не содержит '\'.
  S-11 отзыв сессий: в сессию писать fingerprint (например, sha256(pass_hash)[:16] +
is_active); в before_request
       для авторизованных — один SELECT по id, при несовпадении/блокировке →
session.clear().
  S-02/S-12 привилегии: заменить прямой вызов psql (app.py:1343-1346) на вызов
/usr/local/bin/1c-db-size
       <имя…> с валидацией имён ^[a-z0-9_]{1,63}$ внутри скрипта; подготовить sudoers
только для нужных скриптов;
       подготовить unit с отдельным пользователем (1c-portal), ProtectSystem=strict,
ProtectHome=yes,
       PrivateTmp=yes, NoNewPrivileges нельзя из-за sudo — зафиксировать это в комментарии.
  S-05 1c-sandbox.sh: rac infobase create … --scheduled-jobs-deny=on --sessions-deny=off;
проверить возможность
       NOSUPERUSER для роли копии; password_encryption='scram-sha-256' при поддержке
pg_hba.
  S-10 секреты: EnvironmentFile=/etc/1c-portal/portal.env (0640 root:<portal-user>) вместо
Environment= в unit;
       maintenance.notify — заменить curl с URL в argv на urllib (или --config файл);
шифровать auto_pwd, backup_pass,
       vpn_config ключом из .secret_key (cryptography.Fernet или HMAC-based, если без
зависимостей);
       подготовить инструкцию по ротации хука Mattermost.
Этап B — надёжность и производительность портала:
  C-02 прогрев: _start_warmer выполняет warm_* и maintenance.tick() внутри with
app.app_context(); _cached()
       не должен читать request.args (передавать force параметром).
  C-03/C-04/C-05 кэш: один фоновый коллектор (отдельный процесс/поток только в одном
воркере — например,
       через файловый lock) пишет JSON в /run/1c-portal/state.json; дедупликация обновлений
(Lock на ключ);
       1c-sessions-count: один 'rac session list --cluster' + один 'infobase summary list',
группировка awk.
  C-06 SQLite: при открытии соединения PRAGMA journal_mode=WAL (один раз при init_db),
synchronous=NORMAL,
       busy_timeout=10000; индексы audit(action, actor), audit(action, details),
visits(username, id);
       ретенция audit/visits/client_sessions (параметр, по умолчанию 180 дней) в прогреве
раз в сутки.
  C-07 логирование: logging в stderr (journald) уровня WARNING вместо except: pass в
ключевых местах;
       audit(action, details, actor=None) — явный actor из фоновых потоков.
  C-11 /jobs/active: опрос раз в 5 с только пока active=true, иначе backoff до 60 с;
исключить /jobs/*, /api/*,
       /static/* из _touch_visit.
  P-04 zero-downtime: ExecReload=/bin/kill -HUP $MAINPID в unit; deploy.sh использует
systemctl reload;
       gunicorn --graceful-timeout 30.
  P-05 бэкап: pg_dump -Fd -j 3 --compress=zstd:3 во временный каталог на /var/lib/pgpro,
затем перенос на CIFS;
       проверка кода возврата и pg_restore -l; запись в <имя>.tmp с переименованием.
Этап C — архитектура (отдельная ветка, после согласования):
```

```
  P-01 вынести трафик /WEB/ из Python: эндпоинт /auth/check (200/403 по cookie и slug из X-
Original-URI),
       nginx auth_request на .15 и прямой proxy_pass на Apache-vhost 192.168.0.5:8082
(Require ip 192.168.0.15);
       баннер регламентных работ через sub_filter на стартовой странице; подготовить
конфиги, не применять.
  C-01 разбиение app.py на blueprints: auth, gateway, admin, sandbox, jobs, settings,
maintenance; services/rac.py,
       services/pg.py, services/cache.py; единая функция run_privileged(script, *args,
timeout).
  C-09 тесты и CI: tests/ (pytest), ruff, shellcheck, bandit; GitHub Actions workflow без
секретов.

КРИТЕРИИ ПРИЁМКИ
- pytest зелёный; новые тесты покрывают каждую находку этапа A (негативные сценарии).
- flask routes: ни один клиентский маршрут не выполняет cluster-wide изменений PostgreSQL.
- После деплоя (по команде пользователя) read-only проверки: curl с
'/WEB/<своя>/../<чужая>/' → 404;
  журнал без Traceback за 24 ч; p90 '/' < 300 мс по access-логу (%D); отсутствие 'database
is locked'.
- В отчёте пользователю: что сделано, где проверено, что осталось; версия не менялась, пуш
не выполнялся.
```

3. Инвентаризация

3.1. Код (репозиторий)

Приложение	`app.py` — Flask 3, ~3140 строк, 60+ маршрутов; `maintenance.py` (окна регламентных работ), `metrics.py` (CPU/RAM/сеть/диски из /proc), `1c-maintenance.py` (CLI).
Хранилище	SQLite `portal.db`: users, bases, access, audit, visits, client_sessions, metrics, settings, login_fails, qr_logins, jobs, sandboxes, maintenance, migrations. Схема мигрируется в `init_db()` при импорте модуля.
Аутентификация	PBKDF2 (Werkzeug), подписанная cookie-сессия 8 ч, CSRF-токен в сессии, 5 неудач → блок 15 мин по (логин, IP) + Mattermost, TOTP 2FA (только администратор может включить), вход по QR.
Привилегированные операции	`sudo -n /usr/local/bin/1c-*`: sessions-count, sessions-list, kick-sessions, jobs-deny, base-state, licenses, create-base, republish-base, pg-update-mode, apply-settings, restart-schedule, safe-restart, monitor, backup.sh, sandbox, backup-info; прямой `sudo -u postgres psql` (размеры баз); `sudo systemctl reboot`.
Шлюз 1С	`/WEB/<slug>/<rest>`: проверка сессии и прав → `http.client` к 127.0.0.1:8081, потоковая передача, вставка `maint.js` на стартовую страницу.

Скрипты	23 bash-скрипта с `set -uo pipefail`; SQL к PostgreSQL через `su -s /bin/bash postgres -c "psql …"` со строковой интерполяцией имён баз.
Деплой	`deploy.sh`: apt python3-flask gunicorn, копирование в `/opt/1c-portal`, unit `1c-portal.service` (www-data), vhost-ы Apache. Тестов и CI нет; зависимости не зафиксированы.
Документация	`docs/`: security.md, 1c-performance-plan.md, proxy-nginx.md, backup.md, sandbox-plan.md, new-server-plan.md, client-*.md, index.html.

3.2. Сервер 192.168.0.5 (снимок 08.11.2026 21:08 MSK, только чтение)

Платформа	Ubuntu 24.04.4 LTS, ядро 6.8.0-147, BM KVM, 12 vCPU Intel Xeon L5520 @ 2,27 ГГц, RAM 20 ГБ (used 9,8 / shared 4,3 / available 10,2), swap-файл 3,8 ГБ (занято 57 МБ), swappiness=1, THP=madvise. Uptime 1 д 19 ч, load 1,3.
Диски	`/` 49 ГБ ext4 — занято 69 % (32 ГБ); `/var/lib/pgpro` 40 ГБ ext4 — 40 % (15 ГБ); `/mnt/1cbackup` CIFS //192.168.0.79/backup — 470 ГБ, 50 %, опции `soft, uid=postgres, gid=www-data, file_mode=0660`. Оба тома `noatime`, планировщик none/mq-deadline (SSD/virtio).
1С	8.3.27.2342 (ragent/rmngr/rphost под usr1cv8, `-seclev 0`), ras на 0.0.0.0:1545 от root; старая платформа 8.3.27.1989 (2,4 ГБ) не удалена. **rphost: 101 % CPU, RSS 4,4 ГБ, 1 ч 14 мин** при 3–4 простаивающих сеансах. HASP (hasplm/aksusbd) активны, порт 1947 на всех интерфейсах.
PostgreSQL	Postgres Pro 1C 16.11, данные `/var/lib/pgpro/1c-16/data`; слушает **127.0.0.1:5432 и 192.168.0.5:5432**; checkpointer RSS 2 ГБ; роли вида `usr_<база>`; базы: dgek, goncharov, monesbuh, test, demo + песочницы `sb_*`.
Портал	gunicorn 20.1.0, `--worker-class gthread --workers 2 --threads 32 --timeout 900`, User=www-data, `PORTAL_WARM=1`, `PORTAL_VERSION=2.0.0`, `PrivateTmp=false`, `NoNewPrivileges=no`, хук Mattermost в `Environment=` unit-файла (644). Flask 3.0.2 / Werkzeug 3.0.1 / Jinja2 3.1.2 / Python 3.12.3 из apt. MainPID с 19:57, NRestarts=0, но в журнале 452 строки «Booting worker» (≈226 перезапусков за время жизни журнала).
Apache	2.4.58, mpm_event (ThreadsPerChild 25, MaxRequestWorkers 150), KeepAlive 5 с, `ServerTokens OS`, `ServerSignature On`, mod_status загружен; vhost `*:80` `ServerAlias *` → ProxyPass на 8080 (с `%D` в LogFormat — удобно для измерений); vhost 127.0.0.1:8081 с 7 публикациями (включая сироту `montesbuh` и каталог `dgek2026` владельца dgek), `AllowOverride All`; остатки `ProxyPass /dgek/ !`, `/dgek2/ !`.

Сеть/доступ	Слушают: 22, 80 (*), 1540/1541/1545/1560 (0.0.0.0), 1947 (*), 5432 (lo + LAN IP), 8080/8081 (lo). ufw и fail2ban активны (правила не видны без root; есть jail `1c-portal.conf`). SSH: `PermitRootLogin yes` в основном файле, переопределён `99-hardening.conf` → prohibit-password; парольная аутентификация не отключена. 65 пакетов ожидают обновления, unattended-upgrades активен, перезагрузка не требуется.
Таймеры/cron	`1c-monitor.timer` (10 мин), `1c-backup-tick.timer` (каждую минуту), `/etc/cron.d/1c-portal-backup` (содержимое не прочитано — возможное дублирование с таймером), apt/fstrim. `1c-maintenance-tick.timer` на сервере отсутствует — tick выполняется только внутри gunicorn (в обоих воркерах).
Логи	Сегодня: 18 175 запросов к порталу, из них 13 154 (72 %) — `/WEB/`, 1 634 (9 %) — `/jobs/active`; 11 × 500, 45 × 402, 27 × 403. Журнал портала за 7 дней: 571 строка с ошибками, 7 × «database is locked», Traceback-и `BuildError 'index' / _dt_ru()` в `error.html` (08.11 02:59), `Worker failed to boot` (07.11 21:52). Apache: «Connection refused 127.0.0.1:8080» при перезапусках портала (14:16, 19:48). Journald 742 МБ, apt-кэш 243 МБ.
Бэкап	Ночью 5 баз по ~1 ГБ, `pg_dump -Fc` ≈ 2,5 мин на базу (≈7 МБ/с), на CIFS; индекс копий `backup-files.json` 644.

4. Безопасность

Шкала: Критично — эксплуатируется клиентом и даёт доступ к чужим данным; Высокий — нарушение границ доверия/целостности; Средний — требует условий или даёт ограниченный эффект; Низкий — укрепление; Инфо.

S-08 Критично Обход прав доступа к базам через сегменты «..» в шлюзе /WEB/

app.py:466-481 (`web_gate`), Apache vhost 127.0.0.1:8081

Проверка прав выполняется для `slug` из URL, а `rest` (`<path:rest>`) без изменений добавляется к пути запроса к Apache 8081. Werkzeug не удаляет сегменты `..` (включая `%2e%2e` и `..%2F`), а Apache нормализует путь перед сопоставлением `Alias`. Подтверждено на сервере (in-process тест маршрутизации на Flask 3.0.2 и GET к 8081): `/WEB/demo/../test/` → `slug='demo', rest='../test/'`; Apache 8081 отвечает 200 стартовой страницей базы `test`. Клиент с доступом к одной базе получает веб-клиент любой опубликованной базы (включая чужие песочницы); аудит пишет только `slug` своей базы.

Исправление. В `web_gate`: разбить `rest` по `/`, отклонять сегменты `''`, `'.'`, `'..'`, символы `\`, `\x00`; альтернативно `norm = posixpath.normpath('/WEB/%s/%s' % (slug, rest))` и требовать `norm == '/WEB/%s' % slug or norm.startswith('/WEB/%s/' % slug)`, иначе `_web_denied(slug, 'not-found', 404)`. То же для `web_gate_redirect`. На Apache 8081 дополнительно можно включить отдельные `<Location>` с `Require all denied` для всего, кроме `/WEB/<slug>/`. Покрыть тестами (4 варианта кодирования).

S-04 Высокий Песочница: восстановление дампа чужой базы

app.py:2663-2735 (`base_sandbox_create`), проверка на строке 2699; 1c-sandbox.sh:66-118

Поле формы `dump` проверяется только условием `dump.startswith('/mnt/1cbackup/') and os.path.exists(dump)` . Список копий в интерфейсе фильтруется по `db_name` клиента, но POST принимает любой существующий файл каталога. Имена дампов предсказуемы (`<база>-YYYYMMDD-HHMM.dump`), список всех баз виден в логах заданий (S-06). Результат: клиент базы А получает полную копию базы В с правами на новую публикацию. Дополнительно путь попадает в строку `su … -c "pg_restore … '$DUMP'"` (интерполяция в shell), что при появлении файла с кавычкой в имени даст выполнение команд от postgres.

Исправление. Сопоставлять `os.path.basename(dump)` со списком `_sandbox_copies(row['db_name'])` , проверять `os.path.realpath(dump)` внутри `SANDBOX_DIR` и regex `^[a-z0-9_]+-\d{8}-\d{4}\.dump$` ; передавать в скрипт только имя файла, а каталог добавлять в скрипте. В скриптах — массивы аргументов и `psql -v name=…` / `format('%I')` вместо интерполяции.

S-03 Высокий Клиент переводит весь кластер PostgreSQL в небезопасный режим

app.py:2277-2298 (`portal_base_update_mode`), 1c-pg-update-mode.sh:37-61

Маршрут доступен любому клиенту с доступом к базе. Скрипт выполняет `ALTER SYSTEM SET fsync='off'` , `full_page_writes='off'` , `synchronous_commit='off'` , `autovacuum='off'` — это параметры *всего* экземпляра, а не одной базы. При сбое питания/ядра возможна необратимая порча всех баз; выключенный autovacuum ведёт к распуханию и риску wraparound. Автоматического возврата нет (только вручную или по окончании окна регламентных работ, если режим включали через него). В UI клиента это выглядит как безобидный переключатель «ускорение обновления».

Исправление. Удалить клиентский маршрут (оставить `/admin/bases/<bid>/update-mode`); в скрипте не трогать `fsync` и `full_page_writes` (основной выигрыш даёт `synchronous_commit=off` + `maintenance_work_mem` + `max_wal_size` + `checkpoint_timeout`); таймер авто-возврата (например, через 6 ч) с уведомлением; запись в аудит и Mattermost при включении; на время режима показывать баннер администратору.

S-01 Высокий Подделка X-Forwarded-For обходит защиту от подбора пароля

app.py:293-298 (`_client_ip`), 42 (`ProxyFix(x_for=1)`), 576-607 (`_login_blocked/_login_fail`)

Цепочка прокси: nginx (.15) добавляет IP клиента в конец `X-Forwarded-For` , Apache (.5) добавляет 192.168.0.15. `_client_ip()` берёт *первый* элемент заголовка — его целиком контролирует клиент. Блокировка ведётся по паре (логин, IP): меняя заголовок в каждом запросе, атакующий никогда не блокируется; журнал входов, уведомления в Mattermost и jail fail2ban (если он парсит IP из журнала портала) получают ложные адреса. Вероятная причина решения: с `x_for=1` `request.remote_addr` возвращал адрес nginx.

Исправление. `ProxyFix(app.wsgi_app, x_for=2, x_proto=1, x_host=1)` (две доверенные прокси) и использовать `request.remote_addr` ; удалить ручной разбор. Дополнительно ограничить попытки по логину независимо от IP (например, 20 неудач за 15 мин на логин) и по IP независимо от логина. Тест: запрос с `X-Forwarded-For: 1.2.3.4` через цепочку должен записать реальный IP.

S-02 Высокий sudo на psql для www-data: произвольный SQL суперпользователя из веб-процесса

app.py:1333-1350 (`_base_sizes_fetch`), сервер `/etc/sudoers.d/1c-psql` (60 байт, содержимое не читается без root; размер совпадает со строкой `www-data ALL=(postgres) NOPASSWD: /opt/pgpro/1c-16/bin/psql`)

Правило позволяет любому процессу пользователя www-data — портал, Apache, модуль 1cws, обрабатывающий внешний трафик, — выполнить `psql` с произвольными аргументами от имени `postgres` : чтение/изменение всех баз 1С, `COPY … TO PROGRAM` (команды ОС от postgres). Ошибка в любом компоненте под www-data превращается в полный доступ к данным. Экранирование имён баз в SQL (`'` → `''`) корректно, но `db_name` задаётся администратором без валидации (S-17).

Исправление. Портал вызывает `sudo -n /usr/local/bin/1c-db-size <имя>…` (скрипт уже есть на сервере), скрипт проверяет имена regex и выполняет фиксированный запрос; правило на `psql` удалить. Запустить gunicorn под отдельным пользователем (`1c-portal`) и выдать sudo-правила только ему; Apache оставить без sudo. В долгосрочной перспективе — привилегированный helper (systemd-служба с unix-socket и allow-list команд) вместо sudo.

S-05 Высокий Песочница выполняет регламентные задания боевой базы; роли SUPERUSER

1c-sandbox.sh:76-82, 113-116; 1c-create-base.sh:40-42; 1c-prepare-base.sh:22

`rac infobase create` для копии вызывается без `--scheduled-jobs-deny=on` : восстановленная копия с теми же настройками выполняет обмены, рассылки писем, интеграции с внешними системами — от имени клиента и с его данными. Роли PostgreSQL для баз и копий создаются с `SUPERUSER` (комментарий ссылается на практику 1С для С-функций) и `password_encryption='md5'` . Пароль роли передаётся в командной строке `rac … --db-pwd=` (виден в `ps` на время вызова). Временные файлы `/tmp/sandbox-restore.log` , `/tmp/sb-users.csv` с фиксированными именами (при `PrivateTmp=false`).

Исправление. Добавить `--scheduled-jobs-deny=on` (и при желании `--denied-message`); в карточке песочницы показывать «регламентные задания отключены»; проверить на тестовой базе работу 1С с ролью без SUPERUSER после создания (`ALTER ROLE … NOSUPERUSER`); перейти на scram-sha-256 (требует согласованного `pg_hba.conf`); `mktemp` для временных файлов.

S-06 Средний IDOR: логи любых заданий доступны любому авторизованному пользователю

app.py:3052-3080 (`job_status` , `job_active`), 2997-3035 (`_job_read`)

`/jobs/<jid>/status` защищён только `login_required` : клиент перебором id получает логи архивации (имена всех баз, размеры, пути на CIFS) и логи песочниц других клиентов. `/jobs/active` показывает всем заголовок текущего задания.

Исправление. Для роли client — только задания, где `user_id = session['uid']` или `ref` ссылается на песочницу пользователя; остальным 404. В `/jobs/active` для клиентов отдавать только собственные задания.

S-07 Средний Open redirect в параметре next

app.py:754-755 (`login`), 2451 (`login_2fa`)

Проверка `nxt.startswith('/')` пропускает `//evil.example/…` и `/\evil.example` — браузер уходит на внешний сайт после успешного входа (фишинг «введите пароль ещё раз»).

Исправление. `u = urlsplit(nxt); ok = not u.scheme and not u.netloc and nxt.startswith('/') and not nxt.startswith('//') and '\\' not in nxt` .

S-09 Средний Пароль автологина 1С передаётся в URL и хранится открытым текстом

app.py:775-801 (`_open_url` , `open_base`), поле `bases.auto_pwd`

Ссылка `/WEB/<slug>/?N=<пользователь>&P=<пароль>` попадает в access-логи nginx (.15) и Apache (.5), историю браузера и Referer; `auto_pwd` хранится в SQLite без шифрования, доступен через `/admin` в HTML формы.

Исправление. Минимум: не логировать query-string для `/WEB/` (`SetEnvIf Request_URI "[?&]P=" nolog` в Apache; `map` /условный access_log в nginx), заголовок `Referrer-Policy: no-referrer` на ответах портала, шифрование `auto_pwd` ключом из `.secret_key` . Лучше: отдельный пользователь 1С с минимальными правами на каждую базу автологина и предупреждение в UI, что автологин = общая учётная запись.

S-10 Средний Секреты в открытом виде

сервер `/etc/systemd/system/1c-portal.service` (644), app.py:1835-1865 (`_write_settings_json`), maintenance.py:81-84 (`notify` через curl), таблица `settings` / `users.vpn_config`

Хук Mattermost прочитан непривилегированным пользователем из unit-файла — его следует считать скомпрометированным локально. В `portal.db` и `/etc/1c-portal/settings.json` лежат `backup_pass` (SMB), `mm_hook` , `monitor_hook` , `auto_pwd` , приватные ключи WireGuard клиентов (и копии в `/opt/1c-portal/vpn-clients/*.conf`). `maintenance.notify()` передаёт хук в аргументах `curl` — виден в `ps` всем пользователям на время вызова.

Исправление. `EnvironmentFile=/etc/1c-portal/portal.env` 0640 root:<пользователь портала>; шифрование чувствительных полей (Fernet из `cryptography` либо AES через `hashlib` / `hmac` -схему без зависимостей); `urllib` вместо curl; ротация хука после внедрения; `settings.json` 0640 без паролей (пароль SMB — только в `/etc/1c-backup/smb.cred`).

S-11 Средний Сессии не отзываются при блокировке/удалении пользователя или смене пароля

app.py:340-356 (`login_required` , `admin_required`)

Проверяется только `session['uid']` в подписанной cookie. Отключённый пользователь (`is_active=0`) продолжает работать до 8 часов, включая шлюз 1С.

Исправление. В `before_request` для авторизованных — один SELECT по id (`is_active` , fingerprint пароля); при несовпадении `session.clear()` . Либо серверные сессии (таблица sessions) с возможностью «завершить все сеансы пользователя» из админки.

S-12 Средний Общий пользователь www-data и ослабленный unit

сервер unit `1c-portal.service` ; deploy.sh:40-62

Портал, Apache и модуль 1cws работают под одним пользователем; unit на сервере отличается от `deploy.sh` : `PrivateTmp=false` , `NoNewPrivileges=no` , нет `ProtectSystem/ProtectHome/PrivateDevices/RestrictAddressFamilies` . Компрометация любого компонента даёт доступ к `portal.db` , `.secret_key` (подделка сессий) и sudo-правилам.

Исправление. Отдельный пользователь `1c-portal` (группа www-data только там, где нужен доступ к файлам), `ProtectSystem=strict` + `ReadWritePaths=/opt/1c-portal /etc/1c-portal /var/log/1c-portal` , `ProtectHome=yes` , `PrivateTmp=yes` (sudo-скрипты, использующие /tmp, перевести на `mktemp`), `RestrictAddressFamilies=AF_INET AF_INET6 AF_UNIX` . `NoNewPrivileges` несовместим с sudo — задокументировать.

S-13 Средний Целостность резервных копий не контролируется

1c-backup.sh:100-112; монтирование CIFS `soft`

Успех определяется по непустому файлу (`[ -s "$OUT" ]`), код возврата `pg_dump` игнорируется. При сетевом сбое на `soft` -монтировании частичный дамп засчитывается как «ok», мониторинг сообщает «бэкап в норме». Дампы доступны группе www-data (0660) — все веб-процессы читают копии всех баз.

Исправление. Проверять `$?` pg_dump, писать в `<имя>.part` и переименовывать после `pg_restore -l` ; локальный временный каталог + копирование; отдельная группа для чтения копий; периодический тестовый `pg_restore` в песочницу (уже есть механизм).

S-14 Низкий Заголовки безопасности и раскрытие версий

app.py:2154-2185 (`_no_store`), Apache security.conf, nginx на .15

Нет `Content-Security-Policy` , `X-Frame-Options` / `frame-ancestors` , `X-Content-Type-Options` , `Referrer-Policy` ; HSTS на nginx не подтверждён. Apache отдаёт `ServerTokens OS` / `ServerSignature On` , gunicorn — `Server: gunicorn` .

Исправление. В `after_request` для страниц портала (не для `/WEB/`): `X-Frame-Options: DENY` , `X-Content-Type-Options: nosniff` , `Referrer-Policy: same-origin` , CSP с `'self'` (inline-скриптов шаблонов перевести на nonce или отдельные файлы постепенно). `ServerTokens Prod` , `ServerSignature Off` ; HSTS на nginx.

S-15 Низкий Криптографические мелочи

app.py:334-337 (`check_csrf`), 2369-2374 (`_totp_verify`), 2387-2389 (`_backup_codes_new`), 729-731 (`login`)

Сравнение CSRF-токена не константное по времени; TOTP допускает повторное использование кода в окне ±30 с; резервные коды 32 бита (`token_hex(4)`); при несуществующем логине хеш не вычисляется — перечисление пользователей по времени ответа.

Исправление. `hmac.compare_digest` ; хранить `totp_last_counter` и отклонять повтор; `token_hex(5)` и более; «пустой» `check_password_hash` с фиктивным хешем при отсутствии пользователя.

S-16 Низкий Вход по QR: любой пользователь подтверждает любой QR; расхождение с документацией

app.py:659-712 (`qr_status` , `qr_confirm`); docs/security.md

Документация утверждает, что подтвердить может только администратор, код роль не проверяет. Сценарий фишинга: «отсканируйте QR для проверки» → сессия атакующего становится сессией жертвы (в том числе администратора).

Исправление. Для администраторов требовать пароль/TOTP при подтверждении; явное предупреждение на странице подтверждения; синхронизировать документацию с кодом.

S-17 Низкий Ввод администратора без валидации уходит в shell-скрипты, XML и cron

app.py:1220-1300 (`db_name` , `url_path`), 2112-2122 (`cron`), 1538-1541 (`access_save`), 2053-2060 (`mm_hook`); 1c-republish-base.sh:24-40; 1c-restart-schedule.sh:24-27

`db_name` интерполируется в default.vrd (`ib="Srvr=…;Ref=$DBNAME;"`), конфиг Apache и строки `psql` ; расписание cron проверяется только по числу полей; `int(user_id)` без обработки → 500; хук Mattermost без проверки схемы. Эксплуатация требует прав администратора портала, но нарушает принцип минимального доверия к sudo-скриптам.

Исправление. regex `^[a-z0-9_]{1,63}$` для `db_name` , валидация полей cron, `https://` для хуков, `try/except ValueError` ; в скриптах — проверка аргументов на входе.

S-18 Низкий Сетевая поверхность: PostgreSQL на LAN-адресе, RAS/кластер на всех интерфейсах, Apache :80 на всех интерфейсах

сервер: `ss -tln`

PostgreSQL слушает 192.168.0.5:5432 (есть сеансы `postgres dgek 192.168.0.5`), RAS 1545 и порты кластера доступны на 0.0.0.0, Apache принимает HTTP с любого адреса при `ServerAlias *` — весь периметр держится на ufw, правила которого не проверены. Документ security.md заявляет «PostgreSQL — только localhost».

Исправление. `listen_addresses='localhost'`, если внешние клиенты PG не нужны; в vhost `*:80` — `Require ip 192.168.0.15 127.0.0.1` (или `Listen 192.168.0.5:80` + ufw только с .15); убедиться в правилах ufw для 1545/1540-1560/1947 (только VPN/LAN). См. раздел «Что проверить с root».

S-19 Низкий innerHTML с данными сервера в шаблонах

templates/portal.html:581-585, templates/admin.html:1372-1375

Путь/дата копии и ФИО владельца/название базы вставляются в `innerHTML` без экранирования. Источники контролирует администратор (self-XSS), но при появлении пользовательского ввода станет XSS.

Исправление. Строить DOM через `createElement` / `textContent` или функцию `esc()`.

S-20 Инфо Состояние ОС и SSH

`PermitRootLogin yes` в основном файле переопределён `99-hardening.conf` (работает, но хрупко — убрать из основного); `PasswordAuthentication` не отключён; 65 обновлений в ожидании (unattended-upgrades активен); ключ `id_ed25519_dgek1c` авторизован для `dgek`, а не для `root` — для автоматизированного деплоя стоит завести отдельный ключ деплоя с ограничением команд (`command=` в authorized_keys) вместо root.

5. Качество кода и оптимизация

C-01 Монолит и дублирование

app.py целиком; дубли: `login` (716-759) и `_finish_login` (2406-2421); `_pg_update_mode` (1470-1483) и `portal_base_update_mode` (2277-2298); `_sandbox_run_async` (2584-2612) — мёртвый код; `_cached` / `_cached_swr`; повторяющийся блок проверки доступа к базе (7 мест).

Рекомендация. Blueprints: `auth`, `gateway`, `admin`, `sandbox`, `jobs`, `settings`, `maintenance`; модули `services/rac.py`, `services/pg.py`, `services/cache.py`, `services/privileged.py` (`run_privileged(script, *args, timeout)` с allow-list скриптов); декоратор `@base_access_required`; `config.py` для констант и настроек по умолчанию. Делать поэтапно без изменения URL.

C-02 Фоновый прогрев не работает (без контекста приложения)

app.py:901-919 (`_start_warmer`), 830-839 (`_cached` читает `request.args`), 1366-1402 (`warm_sizes` , `warm_counts` используют `db()` → `g`), 1676-1709 (`_sessions_snapshot`)

Поток прогрева вызывает функции, которые обращаются к `flask.g` и `request` ; вне контекста это `RuntimeError` , который проглатывается `except Exception: pass` . Следствие: `basesize:*` , `basestate:*` и `counts:*` для списка всех баз не прогреваются (только частично через `_warm_once`), история `client_sessions` из прогрева не пишется. Пользователь при первом открытии страницы получает пустые размеры/счётчики, а затем SWR-потоки из запросов делают работу, которую должен был делать прогрев.

Рекомендация. `with app.app_context():` вокруг тела цикла; `_cached(key, ttl, fn, force=False)` ; тест, вызывающий `_warm_once()` / `warm_counts()` из отдельного потока с заглушками subprocess.

C-03 Кэш и фоновые задачи на каждый воркер

app.py:828 (`_cache = {}`), 3085-3091, `_cache.clear()` в 12 местах

При 2 воркерах gunicorn существуют два независимых кэша и два потока прогрева: двойные вызовы `sudo rac` каждые 45 с, разные данные на соседних запросах, `_cache.clear()` очищает только один воркер, `maintenance.tick()` выполняется дважды (гонка `kick_all()` /уведомлений). Кэш не ограничен по размеру (ключи `counts:<список slug>` плодятся при изменении состава баз).

Рекомендация. Один коллектор (отдельная systemd-служба `1c-portal-collector` или поток только в воркере, взявшем файловую блокировку) пишет состояние в `/run/1c-portal/state.json` атомарно; воркеры читают файл (mtime-кэш). При текущей нагрузке (~1 запрос/с) допустимо и `--workers 1 --threads 64` как быстрый вариант.

C-04 SWR без дедупликации — шторм процессов rac

app.py:842-866 (`_cached_swr`)

Пока значение устарело, каждый запрос порождает новый поток и новый процесс `sudo rac` (несколько секунд на 2,27 ГГц). Автообновляемые страницы администратора и кабинета дают десятки параллельных rac-процессов → рост CPU и времени ответа всего сервера.

Рекомендация. Словарь «обновление идёт» под `threading.Lock` ; ограниченный `ThreadPoolExecutor(max_workers=2)` ; метрика числа запущенных обновлений в `/healthz` .

C-05 N+1 вызовов rac

1c-sessions-count.sh:24-37; 1c-base-state.sh; app.py `_licenses` , `_sessions_snapshot`

На каждую базу — `infobase info` + `session list` (2N+1 вызовов), плюс отдельные скрипты для состояний, лицензий и списка сеансов — один и тот же `session list` выполняется несколько раз в минуту.

Рекомендация. Один скрипт `1c-cluster-snapshot` : `cluster list` → `infobase summary list` → `session list --cluster` один раз → группировка по `infobase` (awk) → JSON с сеансами, счётчиками, флагами `scheduled-jobs-deny` (требует `infobase info` — кэшировать 5 мин) и лицензиями. Все потребители читают JSON.

C-06 SQLite: режим журнала, блокировки, индексы, ретенция

app.py:125-130 (`db()`), 143-282 (`init_db`), 1597-1673 (`admin_stats`)

Журнал в режиме DELETE (писатель блокирует читателей), таймаут ожидания 5 с по умолчанию, 64 потока + фоновые писатели. За неделю 7 × «database is locked»; распределение времени ответа кабинета (`/` : p50 4 мс, p90 6,1 с, p99 15,2 с) кратно 5 с — характерно для ожидания блокировки. `/admin/stats` (p50 1,36 с) выполняет полные агрегаты по `visits` / `audit` и коррелированные подзапросы без индекса `(action, actor)` ; таблицы растут без ограничения (`_touch_visit` , `audit` на каждое открытие базы).

Рекомендация. `PRAGMA journal_mode=WAL` (однократно), `synchronous=NORMAL` , `busy_timeout=10000` в каждом соединении; индексы `audit(action, actor)` , `audit(action, details)` , `visits(username, id)` ; ретенция (180 дней) и `VACUUM` раз в сутки из коллектора; агрегаты статистики считать в коллекторе и хранить в таблице `stats_daily` . Измерять: логировать запросы > 1 с с длительностью этапов.

C-07 Ошибки глотаются; аудит из фоновых потоков теряется

более 40 конструкций `except Exception: pass` ; app.py:284-290 (`audit` обращается к `session`), 2942-2995 (`_job_start.work`), 2740-2790 (`sandbox_delete.done`)

`audit('job_…')` , `audit('sandbox_delete')` вызываются из потоков без request-контекста → `RuntimeError` внутри `audit` → запись не создаётся. Диагностика проблем (например, C-02) невозможна без логов.

Рекомендация. `import logging; log = logging.getLogger('portal')` , `log.warning(..., exc_info=True)` в catch-блоках; `audit(action, details, actor=None)` ; единый обработчик 500 с записью в журнал.

C-08 Шлюз /WEB/: соединения, таймауты, память

app.py:466-543 (`web_gate`), unit `--timeout 900`

Новое TCP-соединение к 8081 на каждый запрос; один таймаут 300 с на все типы запросов; стартовая страница читается целиком и перекодируется ради вставки `<script>` ; `Content-Length` не пробрасывается (chunked везде). При 32 потоках на воркер длинные `poll` -запросы 1С (60 с) занимают потоки.

Рекомендация. Пул соединений (`urllib3.HTTPConnectionPool` с keep-alive) или `http.client` с повторным использованием; connect-timeout 5 с, read-timeout 120 с (300 с только для печатных форм по признаку пути); пробрасывать `Content-Length` при потоковой передаче; главное — P-01.

C-09 Тесты, CI, зависимости

Нет `tests/` , нет workflow в `.github/` (только шаблоны), зависимости из apt без фиксации версий, `shellcheck` не применяется (в скриптах есть интерполяция в `su -c`). Есть полезный pre-commit hook против секретов.

Рекомендация. pytest + `app.test_client()` с временной SQLite и заглушкой `subprocess.run` ; `ruff` , `bandit -r .` , `shellcheck *.sh` ; GitHub Actions (без секретов); `requirements.txt` / `constraints` с версиями apt-пакетов для воспроизводимости.

C-10 Расхождение репозитория и сервера

Unit на сервере отличается от `deploy.sh` (timeout 900 против 9000, PrivateTmp, NoNewPrivileges, PORTAL_WARM, хук); на сервере есть скрипты вне репозитория (`1c-publish` , `1c-db-backup.sh` , `1c-import-base` , `1c-db-size` , `1c-base-size`); 8 файлов sudoers против 2 в репозитории; `1c-safe-restart.sh` с захардкоженными `base1 base2` и `1c.dgek.ru` (отключает сеансы не тех баз); публикации-сироты `montesbuh` , каталог `dgek2026` ; старая платформа 8.3.27.1989. В AGENTS.md указана версия на сервере 1.9.5, фактически unit содержит 2.0.0.

Рекомендация. Каталог `deploy/` в репозитории с unit, sudoers, apache-конфигами, скриптами; `deploy.sh` с режимами `check` (diff/sha256 без изменений) и `apply` ; список баз для safe-restart брать из `portal.db` /rac; удалить сироты после подтверждения.

C-11 Опрос /jobs/active и учёт визитов

templates/portal.html:692-706, templates/admin.html:1451-1465, app.py:302-316 (`_touch_visit`)

Каждая открытая вкладка опрашивает `/jobs/active` каждые 5 с (1 634 запроса за день, 9 % трафика портала) независимо от наличия заданий; каждый запрос попадает в `_touch_visit` и продлевает визит — статистика «времени в портале» отражает время открытой вкладки.

Рекомендация. Опрос 5 с только при `active=true` , иначе 60 с; `_touch_visit` пропускать для `/jobs/` , `/api/` , `/static/` ; либо Server-Sent Events.

C-12 Мелочи

`query_string.decode('latin-1')` корректно для побайтовой передачи — добавить комментарий; `_cached` зависит от `request` ; `time.strptime` / `time.timezone` в `qr_status` (ошибка при переходе на летнее время — для MSK неактуально); пароль администратора при первичной инициализации печатается в journald (257-265) — лучше в файл 0600; `error.html` использует `url_for('index')` — при неполном импорте модуля даёт 500 внутри обработчика 500 (наблюдалось 08.11 02:59).

6. Производительность

6.1. Измерения (access-лог Apache за 08.11.2026, поле %D)

Класс запросов	n	p50	p90	p99	max
Все запросы портала	18 175	16 мс	118 мс	2,5 с	245 с
`/WEB/*` (веб-клиент 1С через портал)	13 154	21 мс	130 мс	1,66 с	60 с
… из них статика 1С (js/css/png)	1 069	14 мс	166 мс	625 мс	8,2 с
… e1cib/ping, poll	570	15 мс	37 мс	1,0 с	10 с
`/` (кабинет клиента)	338	4 мс	**6,1 с**	**15,2 с**	43,6 с
`/admin*`	468	55 мс	1,36 с	15,8 с	245 с (действия с rac/psql)
`/admin/stats`	58	1,36 с	—	—	—
`/login`	357	6 мс	266 мс	2,9 с	10 с

`/jobs/active`	1 634	5 мс	6 мс	27 мс	4,5 с
`/static/*`	1 611	6 мс	28 мс	42 мс	83 мс

Пиковая нагрузка 14:00–17:00 — до 3 900 запросов/час (≈1,1 запрос/с). HTTP-слой не является узким местом по пропускной способности; проблема — в хвостах (блокировки SQLite, ожидание rac/psql) и в стоимости одного ядра для 1С.

P-01 Вынести трафик веб-клиента 1С из Python

Сейчас каждый из 13 тыс. запросов в день проходит nginx → Apache → gunicorn (Python, GIL, 64 потока) → Apache 8081 → 1С. Это добавляет два hop-а, занимает потоки на long-poll (60 с) и печать (до 300 с), и делает портал единой точкой отказа для работы в 1С (при перезапуске портала веб-клиент падает — в логах Apache «Connection refused» 14:16 и 19:48).

Схема. nginx на .15: `location /WEB/ { auth_request /_portal_auth; proxy_pass http://192.168.0.5:8082; }` ; `location = /_portal_auth { internal; proxy_pass http://192.168.0.5:80/auth/check; proxy_set_header X-Original-URI $request_uri; proxy_pass_request_body off; }` . Портал: `/auth/check` читает cookie сессии и slug из `X-Original-URI` (с нормализацией — S-08), отвечает 200/403/401 и кэшируется nginx 30 с по cookie+slug. Apache на .5: новый vhost `192.168.0.5:8082` с теми же публикациями и `Require ip 192.168.0.15` . Баннер регламентных работ — `sub_filter '</body>' '<script src="/static/maint.js" defer></script></body>'` только для `location ~ ^/WEB/[^/]+/$` . Ожидаемый эффект: p50 /WEB/ ≈ время 1С без надбавки Python, устойчивость к перезапускам портала, освобождение потоков gunicorn. Сначала добавить `%D` в LogFormat vhost-а 8081, чтобы измерить вклад портала до/после.

P-02 Хвосты кабинета и админки

См. C-06 (WAL, busy_timeout, индексы, ретенция) и C-02/C-04 (прогрев и дедупликация). Цель: `/` p90 < 300 мс, `/admin?tab=bases` p90 < 500 мс, `/admin/stats` p50 < 300 мс. Контроль по полю %D в access-логе.

P-03 Коллектор rac и снижение числа процессов

Один процесс каждые 60 с: один `session list` , один `infobase summary list` , размеры баз одним запросом через обёртку (S-02), лицензии раз в 5 мин. Запись JSON атомарно. Убирает десятки `sudo` + `rac` процессов в минуту на 2,27 ГГц ядрах (сейчас по журналу sudo — вызовы каждые ~45 с из каждого воркера плюс SWR-потоки из запросов).

P-04 Перезапуск без простоя

В unit добавить `ExecReload=/bin/kill -HUP $MAINPID` , в deploy — `systemctl reload 1c-portal` (gunicorn перезапускает воркеры по очереди); `--graceful-timeout 30` ; на Apache `ProxyPass … retry=0` уже есть. После P-01 перезапуск портала перестанет влиять на веб-клиент 1С.

P-05 Резервное копирование быстрее и надёжнее

Текущие ≈2,5 мин на 1 ГБ базы: однопоточное сжатие zlib в `pg_dump -Fc` на медленном ядре плюс запись на CIFS. PostgreSQL 16 поддерживает `--compress=zstd:3` / `lz4` и параллельный формат каталога `-Fd -j 3` . Дамп во временный локальный каталог (на `/var/lib/pgpro` свободно 23 ГБ), проверка `pg_restore -l` , затем перенос на CIFS (`tar` или каталог). Ожидаемо в 3–5 раз быстрее при меньшей нагрузке на CPU в рабочее время (ручные запуски днём наблюдались 15:08–18:38).

P-06 Сервер 1С: rphost 100 % одного ядра

В момент снимка (воскресенье 21:08) `rphost` потреблял 101 % CPU и 4,4 ГБ RSS более часа при 3–4 простаивающих сеансах — вероятно, регламентные/фоновые задания (в том числе баз `test`, `demo`, песочниц). На CPU с низкой частотой это напрямую удлиняет отклик интерактивных сеансов.

Действия. `rac session list` с `cpu-time-current` / `current-service-name`, технологический журнал (события CALL/SCALL по длительности); по умолчанию `scheduled-jobs-deny=on` для test/demo/песочниц (S-05); ограничить фоновые задания по расписанию; перезапуск rphost в нерабочее время (уже в docs/1c-performance-plan.md). Рассмотреть ограничение памяти рабочего процесса в настройках кластера.

P-07 PostgreSQL (проверить с root)

Конфигурация не читается без root. По памяти процессов `shared_buffers` ≈ 4 ГБ (20 % RAM — норма для 1С). Проверить: `effective_cache_size` ≈ 12 ГБ, `work_mem` 32–64 МБ, `maintenance_work_mem` 1 ГБ, `random_page_cost=1.1` (SSD), `checkpoint_completion_target=0.9`, `wal_compression=on`, `huge_pages=try`, `max_connections` (сейчас много idle-подключений от 1С), настройки autovacuum для 1С (`autovacuum_vacuum_scale_factor=0.02`, `autovacuum_analyze_scale_factor=0.01`, `autovacuum_max_workers=4`), `pg_stat_statements` и `track_io_timing` для поиска тяжёлых запросов; регулярный `REINDEX` / `VACUUM ANALYZE` в окно регламентных работ (есть механизм окон).

P-08 ОС и место на диске

Корень 69 %: удалить платформу 8.3.27.1989 (2,4 ГБ), `journalctl --vacuum-size=200M` + `SystemMaxUse=200M` (742 МБ), `apt-get clean` (243 МБ); установить обновления в окно работ; `sysstat` уже активен — использовать `sar` для истории CPU/IO. Параметры VM (swappiness=1, THP madvise, noatime) настроены хорошо.

P-09 Apache

`KeepAliveTimeout 5` → 15–30 с (nginx держит upstream keepalive 32); `ProxyPass … enablereuse=on keepalive=On` к 8080 и 8081; `AllowOverride None` в публикациях (экономит stat .htaccess на каждый запрос); удалить `ProxyPass /dgek/ !`, `/dgek2/ !` и публикации-сироты; `ServerTokens Prod`; `mod_status` — только с 127.0.0.1 или отключить; после P-01 — vhost 8082.

P-10 Фронтенд портала

Статика версионируется (`?v=2.0.0`) и кэшируется 30 дней — хорошо. `brand.png` запрошен 487 раз за день (проверить, что ответы 304/кэш, а не 200 с телом; добавить `immutable`). Иконки Lucide (`vendor/lucide.js`) грузятся 156 раз — рассмотреть инлайн-спрайт нужных иконок.

P-11 Железо

Главный ограничитель интерфейса 1С — Xeon L5520 2,27 ГГц (2009 г.): всё однопоточное (rphost, pg_dump, Python) упирается в частоту. План переезда на i7-10700K (docs/new-server-plan.md) остаётся основным рычагом ускорения 1С (×3 на поток); меры этого документа ускоряют портал и снижают риск, но не заменяют переезд.

7. План внедрения

Этап	Содержание	Результат/критерий

A. Срочно	S-08, S-04, S-03, S-01, S-06, S-07 в коде; S-02 (обёртка вместо psql), S-05 (`--scheduled-jobs-deny=on`), S-10 (EnvironmentFile, ротация хука), S-11; тесты на каждую находку.	Клиент не выходит за пределы своих баз; cluster-wide действия только у администратора; журнал IP достоверен.
B. Надёжность	C-02, C-03/C-04/C-05 (коллектор), C-06 (WAL, индексы, ретенция), C-07, C-11, P-04, P-05, S-13, S-12 (отдельный пользователь, hardening unit), C-10 (deploy/ в репозитории, check/apply).	Нет «database is locked»; p90 кабинета < 300 мс; перезапуск без ошибок у пользователей; бэкап проверяется pg_restore -l.
C. Архитектура	P-01 (auth_request), C-01 (blueprints), C-09 (CI), S-14 (заголовки/CSP), S-09 (шифрование и логи), S-18 (сетевой периметр), P-07/P-09 на сервере.	Веб-клиент 1С не зависит от Python; код разбит на модули с тестами; периметр задокументирован и проверен.
D. Сервер 1С	P-06 (анализ rphost, регламентные задания), P-08 (место, обновления), затем переезд по new-server-plan.md.	Нет постоянной 100 % загрузки ядра; корень < 50 %; обновления установлены.

8. Проверки и приёмка

8.1. Негативные тесты (pytest, app.test_client)

```
# S-08: клиент с доступом к demo не достигает test
for url in ('/WEB/demo/../test/', '/WEB/demo/..%2Ftest/', '/WEB/demo/%2e%2e/test/'):
    assert client.get(url).status_code == 404
# S-04: чужой дамп отклоняется
r = client.post('/bases/1/sandbox', data={'csrf': tok, 'dump': '/mnt/1cbackup/other-20261108-
0400.dump', 'ack': '1'})
assert 'Копия не найдена' in flashes(r)
# S-03: клиентский маршрут удалён
assert client.post('/portal/bases/1/update-mode', data={'csrf': tok}).status_code in (403, 404,
405)
# S-01: поддельный XFF не влияет на IP
client.post('/login', data=bad_creds, headers={'X-Forwarded-For': '1.2.3.4, 10.0.0.1'})
assert audit_last_ip() == '127.0.0.1'
# S-06: чужое задание недоступно
assert client.get('/jobs/%d/status' % foreign_job).status_code == 404
# S-07: open redirect
assert client.post('/login?next=//evil.example',
data=good_creds).headers['Location'].endswith('/')
```

8.2. Read-only проверки на сервере (до и после деплоя, выполняются по команде пользователя)

```
# хвосты времени ответа по access-логу (мкс в предпоследнем поле)
awk '$7 == "/" {print $(NF-1)}' /var/log/apache2/1c-portal-access.log | sort -n | awk
'{a[NR]=$1} END {print "p50", a[int(NR*0.5)]/1000, "p90", a[int(NR*0.9)]/1000, "ms"}'
# блокировки SQLite и трассировки за сутки
journalctl -u 1c-portal --since -1d --no-pager -q | grep -ciE 'database is locked|Traceback'
# нормализация пути на внутреннем vhost (должен остаться 200 — это поведение Apache; защита
делается в портале)
curl -s -o /dev/null --path-as-is -w '%{http_code}\n'
http://127.0.0.1:8081/WEB/nonexist/../demo/
# заголовки портала
curl -sI https://1c.dgek.ru/login | grep -iE 'strict-transport|x-frame|content-
security|referrer-policy|x-content-type'
```

9. Что проверить с root (не выполнялось)

- `cat /etc/sudoers.d/1c-*` — подтвердить правило на `psql` (S-02) и полный перечень разрешённых команд; сверить с репозиторием.
- `ufw status verbose` — закрыты ли 5432 (LAN), 1545, 1540–1560, 1947, 80 для всех, кроме .15 и VPN (S-18).
- `/var/lib/pgpro/1c-16/data/postgresql.conf`, `postgresql.auto.conf` (нет ли остатков `fsync=off` после «режима обновления»), `pg_hba.conf` (md5 → scram), `listen_addresses` (P-07, S-03, S-18).
- Права: `ls -la /opt/1c-portal /etc/1c-portal /opt/1c-portal/vpn-clients`; `portal.db`, `.secret_key`, `settings.json` (S-10, S-12).
- `/etc/cron.d/1c-portal-backup` и `1c-backup-tick.timer` — нет ли двойного запуска архивации.
- `/etc/fail2ban/jail.d/1c-portal.conf` и фильтр — какой IP парсится из журнала (после S-01 формат может измениться).
- `/etc/1c-portal/pg-mode.state` — текущее состояние режима обновления (S-03).
- `rac session list` / техжурнал — причина 100 % CPU rphost (P-06).

10. Приложение

10.1. Как собирались данные

- Код: чтение `app.py`, `maintenance.py`, скриптов `1c-*.sh`, `deploy.sh`, sudoers, шаблонов, `docs/`; `git log`.
- Сервер: два bash-скрипта только для чтения от пользователя `dgek` (hostname/lscpu/free/df/ps/ss/systemctl cat/ls/journalctl/awk по логам Apache/curl к 127.0.0.1); in-process тест маршрутизации Flask на тестовом приложении; GET-запросы к 127.0.0.1:8081 (аналог штатного мониторинга). Временные файлы в `/tmp` удалены.
- Не выполнялось: любые записи, sudo, запросы с учётными данными, запросы к внешним адресам.

10.2. Ограничения и допущения

- Содержимое sudoers восстановлено по размеру файла (60 байт) и вызову в коде — требует подтверждения с root.
- Причина хвостов времени ответа кабинета (5-секундные ожидания) — обоснованная гипотеза по распределению и событиям «database is locked»; подтвердить логированием длительности этапов запроса.
- Правила ufw/fail2ban, конфигурация PostgreSQL и права на каталоги портала не проверены.

10.3. Связанные документы репозитория

`docs/security.md` (обновить пункты о QR и «PostgreSQL только localhost»), `docs/1c-performance-plan.md`, `docs/proxy-nginx.md` (основа для P-01), `docs/backup.md` (P-05, S-13), `docs/sandbox-plan.md` (S-04, S-05), `docs/new-server-plan.md` (P-11).

1CSmallControl · аудит и рекомендации · 08.11.2026 · версия портала не изменялась (2.0.0), изменения на сервер не применялись, git push не выполнялся.